

The Mystery Button — Project Summary

Event Date: August 14, 2026 · **Cutoff:** 4:00 PM IST
Prepared for: Informal Team Briefing (Aug 14, 2026)

1. What Is It?

The Mystery Button is a real-time, competitive clicker game built for a college event. Players sign in, smash a big red button as fast as they can, and compete on a live leaderboard — all while a multi-layered anti-cheat system silently keeps things fair.

The top 3 clickers at the event cutoff time win **Mystery Prizes** 🎁.

Key Features

Feature	Description
Big Red Button	A satisfying, animated button with +1 VFX particles and selectable sound effects
Live Leaderboard	Filterable by course and batch, auto-refreshes, medal emojis for top 3
Rank Toasts	Real-time notifications when you break into the top 50 / 25 / 10
Milestone Popups	Unlockable reward images at 1K, 2.5K, 5K, and 10K clicks with Kerala-flavored captions
Event Countdown Timer	Synced to a hardcoded UTC cutoff — locks the game for everyone simultaneously
Bribe the Dev Button	Joke popup with a video — leads to a "buy us chaya" QR code
Gamified CAPTCHA	Shape-click challenge that pops up randomly to catch bots

2. Tech Stack

Frontend

Technology	Version	Role
React	19.x	UI framework — component-based SPA
Vite	6.x	Build tool & dev server (lightning-fast HMR)
React Router	7.x	Client-side routing (Login → Setup → Game → Leaderboard)
Clerk React SDK	5.18.x	Authentication UI (Google OAuth sign-in modal)
Vanilla CSS	—	Custom retro/neo-brutalist styling with Bangers font from Google Fonts

Notable frontend patterns:

- Client-side click throttling at **~15 CPS** (67ms minimum interval between clicks)
- Debounced **batch sync every 3 seconds** (collects clicks locally, sends in one request)
- `keepalive: true` on `beforeunload` to flush unsent clicks when the tab closes
- Periodic rank polling every **10 seconds**
- Per-click `Date.now()` timestamps recorded and sent to server for variance analysis

Backend (Dual-Mode Architecture)

The backend runs in **two modes** from the same codebase:

Mode	Technology	Purpose
Production	Vercel Serverless Functions	Each API route is an independent function (<code>api/*.js</code>) deployed as edge functions
Development	Express.js 4.x	Traditional server (<code>server/index.js</code>) with <code>node --watch</code> for hot-reloading

Both modes import the same shared modules:

- `api/_anticheat.js` — All anti-cheat logic (zero external dependencies)
- `api/_auth.js` — Clerk JWT verification
- `api/_db.js` — Neon database connection

Technology	Version	Role
Vercel	—	Hosting & serverless deployment platform
Clerk Backend SDK	1.21.x	Server-side JWT token verification
Neon (Serverless Postgres)	1.x	Serverless PostgreSQL database (free tier)
Express.js	4.21.x	Local dev server only
dotenv	16.x	Environment variable loading (dev mode)
CORS	2.8.x	Cross-origin headers (dev mode)

Database (Neon PostgreSQL)

Single `players` table with the following schema:

```
CREATE TABLE players (  
  clerk_id      TEXT PRIMARY KEY,  
  name          TEXT NOT NULL,  
  course       TEXT NOT NULL,  
  batch        TEXT NOT NULL,  
  score        INTEGER DEFAULT 0,  
  last_click_sync  TIMESTAMPTZ DEFAULT NOW(),  
  last_click_count INTEGER DEFAULT 0,  
  -- Anti-cheat fields  
  is_shadowbanned  BOOLEAN DEFAULT FALSE,  
  shadow_score     INTEGER DEFAULT 0,  
);
```

```

click_windows      JSONB DEFAULT '[]',
rate_violations    INTEGER DEFAULT 0,
pending_captcha_id TEXT DEFAULT NULL,
pending_captcha_target TEXT DEFAULT NULL,
captcha_issued_at  TIMESTAMPTZ DEFAULT NULL,
captcha_failures   INTEGER DEFAULT 0
);

```

Indexed on: `course`, `batch`, `score` DESC.

API Endpoints

Method	Endpoint	Auth	Description
GET	<code>/api/player</code>	✓ Clerk JWT	Fetch current player's profile & score
POST	<code>/api/player</code>	✓ Clerk JWT	Create/update player (onboarding)
POST	<code>/api/score</code>	✓ Clerk JWT	Submit click batch — full anti-cheat pipeline
POST	<code>/api/verify-captcha</code>	✓ Clerk JWT	Verify CAPTCHA response
GET	<code>/api/leaderboard</code>	Optional	Ranked list with course/batch filters
GET	<code>/api/rank</code>	✓ Clerk JWT	Current player's rank position

3. Anti-Cheat Protocol (v2)

The anti-cheat system is designed with one core philosophy: **cheaters should never know they've been caught**. Every defense layer is server-side and untamperable. Caught players are **shadowbanned** — their game looks and feels completely normal, but their clicks go to a `shadow_score` that doesn't count.

Defense Layers (In Order of Execution)

```

Request → ① Event Cutoff → ② Auth → ③ Cooldown → ④ Timing Clamp
          → ⑤ Shadowban Check → ⑥ Rolling Rate Cap → ⑦ Variance Check
          → ⑧ Random CAPTCHA → ✓ Score Updated

```

Layer ① — Event Cutoff

- **What:** Hardcoded UTC timestamp (`2026-08-14T10:30:00.000Z` = 4:00 PM IST)
- **How:** All score/player-creation endpoints return `403` after cutoff
- **Why:** Prevents late submissions; impossible to bypass by changing client clock

Layer ② — Authentication

- **What:** Every score request requires a valid Clerk JWT
- **How:** `@clerk/backend` verifies the Bearer token server-side

- **Why:** Ties every click to a real Google account — no anonymous spam

Layer ③ — Batch Cooldown

- **What:** Minimum **2-second gap** between score sync batches
- **How:** Compares `last_click_sync` timestamp from the database
- **Why:** Prevents rapid-fire API calls from scripts (returns HTTP 429)

Layer ④ — Timing-Based Clamp

- **What:** Maximum **15 clicks per second** (CPS), capped at **45 clicks per batch**
- **How:** `safeClicks = min(reported_clicks, elapsed_seconds × 15, 45)`
- **Why:** Even if the client reports 1000 clicks, the server only accepts what's physically possible. Elapsed time is capped at 30 seconds to prevent "sleep and claim" attacks.

Layer ⑤ — Shadowban Passthrough

- **What:** Banned players' clicks go to `shadow_score` instead of `score`
- **How:** If `is_shadowbanned = TRUE`, the UPDATE query increments `shadow_score`
- **Why:** The API response looks identical (`{ success: true, accepted: N }`) — the cheater keeps playing happily, never realizing their score is fake

Layer ⑥ — Rolling Rate Cap

- **What:** Max **1200 clicks per 60-second rolling window** (20 CPS sustained)
- **How:** Click batches are stored in a JSONB array (`click_windows`) with timestamps. Old entries are pruned. If the sum exceeds the cap for **3 consecutive checks**, the player is shadowbanned.
- **Why:** Catches sustained autoclicker scripts. The 3-strike rule prevents false positives from short bursts of legitimate fast clicking.
- **Serverless-safe:** No shared memory — all state is in the database row.

Layer ⑦ — Click Variance Analysis

- **What:** Detects inhumanly uniform click timing (autoclicker signature)
- **How:** Computes the **standard deviation** of inter-click intervals from the `timestamps` array sent by the client
- **Triggers only when BOTH conditions are true:**
 1. Stddev of intervals is **< 5 milliseconds** (suspiciously machine-like)
 2. Player's rolling rate is **≥ 80%** of the rate cap
- **Why:** Real humans have natural jitter in their clicks. Autoclickers produce perfectly spaced intervals. The dual condition prevents false positives from macro mice or key-repeat features.

Layer ⑧ — Gamified CAPTCHA

- **What:** Random shape-click challenge with a **5-second timer**
- **How:**
 - ~10% chance of triggering on any score submission
 - Server generates a target shape (circle / star / triangle / square / diamond) among 2–3 decoys

- Client renders an overlay with the shapes at random positions
- Player must tap the correct shape within 5 seconds
- **Stray clicks** (clicking the background, not shapes) count as failure
- **1 retry** allowed; 2nd failure → **silent shadowban**
- Server-side timing check with 2-second network grace period
- **Why:** The final layer catches sophisticated bots that pass rate checks. Even if a bot sends the correct shape type, the stray-click tracking and timeout make it unreliable.

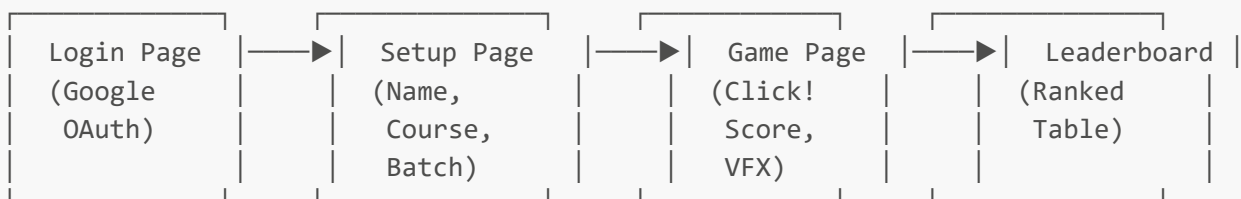
Leaderboard Ghosting 🧟

The leaderboard is also part of the anti-cheat system:

Viewer	What they see
Legitimate player	Only real (non-banned) players and their score
Shadowbanned player	Everyone (including other cheaters) ranked by score + shadow_score — they see themselves on the board with their inflated fake score, reinforcing the illusion that everything is normal

The same ghosting logic applies to the `/api/rank` endpoint — shadowbanned players get a rank computed against all players (real + shadow), so their position never looks suspicious.

4. User Flow



1. **Login** → Player clicks "JOIN THE BATTLE" → Clerk Google OAuth modal
2. **Setup** → First-time players enter name, course, batch (searchable dropdowns covering 24 courses & 3 batches)
3. **Game** → Mash the big red button! +1 VFX particles fly, sound effects play, score updates instantly. Clicks are batched and synced every 3 seconds.
4. **Leaderboard** → View rankings, filter by course or batch. Accessible without auth.

5. Project Structure

```
mystery-button/  
├── api/  
│   ├── _anticheat.js  
│   ├── _auth.js  
│   └── _db.js  
└── # Vercel serverless functions (production)  
    # ← ALL anti-cheat logic lives here  
    # Clerk JWT verification  
    # Neon DB connection
```

```
├── score.js           # POST /api/score
├── player.js         # GET/POST /api/player
├── leaderboard.js    # GET /api/leaderboard
├── rank.js           # GET /api/rank
├── verify-captcha.js # POST /api/verify-captcha
├── client/           # React frontend
│   ├── src/
│   │   ├── pages/
│   │   │   ├── LoginPage.jsx
│   │   │   ├── SetupPage.jsx
│   │   │   ├── GamePage.jsx
│   │   │   └── LeaderboardPage.jsx
│   │   ├── components/
│   │   │   ├── BigRedButton.jsx
│   │   │   ├── CaptchaChallenge.jsx
│   │   │   ├── EventTimer.jsx
│   │   │   ├── PlusOneVFX.jsx
│   │   │   ├── MuteToggle.jsx
│   │   │   ├── RankToast.jsx
│   │   │   ├── SearchDropdown.jsx
│   │   │   └── ProtectedRoute.jsx
│   │   ├── App.jsx      # React Router config
│   │   ├── main.jsx     # Entry point + ClerkProvider
│   │   └── index.css    # All styles (retro/neo-brutalist)
│   └── vite.config.js
├── server/           # Express dev server (local only)
│   ├── index.js        # All routes (mirrors api/*.js)
│   └── db.js           # DB connection with dotenv
├── init.sql          # Database schema
├── vercel.json        # Deployment config
└── package.json      # Monorepo scripts
```

6. Deployment

Component	Platform
Frontend	Vercel (Vite build → static files)
Backend	Vercel Serverless Functions (Node.js)
Database	Neon (Serverless PostgreSQL, free tier)
Auth	Clerk (Google OAuth, free tier)

The `vercel.json` handles routing: API requests go to serverless functions, everything else falls back to `index.html` for client-side routing.

7. Quick Stats

Metric	Value
Lines of anti-cheat code	~300 (all in one file)
External dependencies (anti-cheat)	0 — pure JavaScript
Total API endpoints	6
Frontend pages	4
React components	8
Database tables	1
Supported courses	24
Max legit CPS	15 (client) / 20 (server rolling cap)
CAPTCHA timeout	5 seconds
Event cutoff	Aug 14, 2026 @ 4:00 PM IST

Built with  and questionable amounts of clicking.